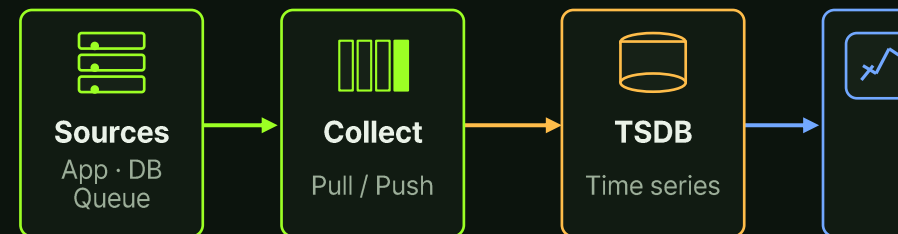


CHAPTER 5

지표 모니터링 및 경보 시스템

Metrics Monitoring & Alerting System

OBSERVABILITY DATA FLOW



metric_name + labels + timestamp + value

수집 → 전송 → 저장 → 질의 → 경보/시각화

핵심 질문

1천만 개 시계열을 지속적으로 수집하면서, 장애를 빠르게 감지하고 운영자에게 신뢰성 있게 전달하려면 어떻게 설계해야 할까?

오늘 답해야 할 설계 질문 다섯 가지

발표 흐름과 핵심 의사 결정

01

무엇을 모니터링할까?

운영 지표만 포함하고
로그·트레이싱·비즈니스
지표는 범위 밖으로 둡니다.

02

어떻게 수집할까?

Pull·Push·Hybrid 중
운영 환경에 맞는 모델을
선택합니다.

03

어디에 저장할까?

쓰기 중심 접근 패턴에 맞춰
시계열 DB와 보존 정책을
설계합니다.

04

어떻게 확장할까?

Collector pool, Kafka,
partitioning, down-
sampling으로 규모를
흡수합니다.

05

어떻게 경보할까?

규칙 평가, 상태 저장,
중복 제거, 재시도와
채널 라우팅을 설계합니다.

문제의 범위를 먼저 좁혀야 올바른 시스템을 설계할 수 있다

내부 운영 지표 모니터링 시스템을 설계한다

포함하는 지표

- CPU·메모리·디스크 사용량
- 서비스의 요청 수와 처리율
- 웹 서버 풀의 실행 인스턴스 수
- 메시지 큐의 적재 메시지 수
- 인프라와 애플리케이션의 운영 상태

이번 설계에서 제외

- 웹 서버 오류·접근 로그 수집
- 분산 추적과 요청 경로 분석
- 매출·전환율 같은 비즈니스 지표
- 외부 고객에게 판매하는 SaaS 기능
- 로그 검색·트레이스 상관 분석

면접에서 확인할 질문

- 사내 플랫폼인가, 외부 SaaS인가?
- 수집할 지표의 종류와 빈도는?
- 데이터 보존 기간과 해상도는?
- 경보 채널은 무엇을 지원하는가?
- 로그와 트레이싱도 포함하는가?

성능보다 먼저 "확장성·지연·신뢰성·유연성"의 우선순위를 합의한다

기능 요구사항과 비기능 요구사항

기능 요구사항

- 다양한 출처에서 지표를 수집한다.
- 시계열 형태로 저장하고 기간 질의를 지원한다.
- 규칙을 평가하여 이메일·전화·PagerDuty·Webhook으로 경보한다.
- 그래프와 대시보드로 추세를 시각화한다.

규모 확장성

- 지표 수와 수집 빈도가 증가해도 수평 확장해야 한다.
- Collector·Queue·Consumer·TSDB를 독립적으로 확장한다.

낮은 응답 지연

- 대시보드와 경보 규칙 질의의 응답 시간을 낮춘다.
- 최근 데이터에 대한 빠른 접근을 우선 최적화한다.

안정성

- 일부 지표 샘플 누락은 허용할 수 있다.
- 하지만 중요한 경보 이벤트와 알림 전달은 놓치지 않는다.

유연성

- TSDB·시각화·경보 제품을 교체할 수 있어야 한다.
- 수집과 처리, 저장과 소비 사이의 결합도를 낮춘다.

운영 원칙

- 모든 컴포넌트를 직접 만들지 않는다.
- 차별화되지 않는 경보·시각화는 검증된 제품을 우선 검토한다.

1천만 개 시계열과 1년 보존 정책이 설계의 중심을 결정한다

책에서 제시하는 대규모 인프라 가정

100M

일간 활성 사용자

서비스 전체 규모

1,000

서버 풀

서비스/클러스터 단위

100

풀당 머신

총 약 100K 머신

100

머신당 지표

CPU·메모리·요청 수 등

10M

시계열

가정치

데이터 보존 정책

최근 7일

원본 해상도 유지

최근 30일

1분 해상도로 Roll-up

최대 1년

1시간 해상도로 Roll-up

시간이 오래될수록 해상도는 낮아지고, 저장 단가는 내려간다

수집 주기가 6배 짧아지면 쓰기량과 저장량도 거의 6배 증가한다

발표용 개략적 규모 추정 — 1천만 시계열 기준

초당 샘플 수

60초 수집



≈ 167K samples/s

10초 수집



1천만 ÷ 수집 간격

10초 수집은 60초 수집보다

처리율·저장량·네트워크가 약 6x

최소 데이터량 예시

샘플당 16바이트로 가정한 순수 값 기준

230GB

60초 수집 / 일

인덱스·복제 제외

1.38TB

10초 수집 / 일

인덱스·복제 제외

- 실제 저장량에는 레이블 인덱스, WAL, 복제본, 메타데이터가 추가됩니다.
- 모든 지표를 같은 주기로 수집하지 않고 중요도별 주기를 차등 적용할 수 있습니다.
- Downsampling과 압축 없이는 1년 보존 비용이 빠르게 폭증합니다.

모니터링 시스템은 다섯 단계의 데이터 파이프라인으로 볼 수 있다

수집 · 전송 · 저장 · 경보 · 시각화



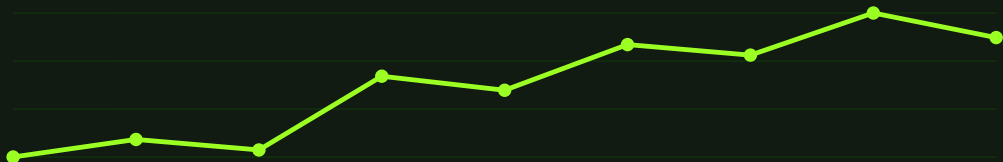
시계열은 "지표 이름 + 레이블 집합"으로 식별된다

타임스탬프가 붙은 값의 연속과 카디널리티 관리

한 개 데이터 포인트

metric_name	cpu_load
labels	{host="i631", env="prod"}
timestamp	1613707265
value	0.29

고유 시계열 ID `cpu_load{host="i631", env="prod"}`



카디널리티 폭발

레이블 조합 수 = 각 레이블 값 개수의 곱



$$5 \times 200 \times 6 \times 100K = 600M \text{ series}$$

- user_id·request_id·email 같은 무제한 값을 레이블로 넣지 않습니다.
- 고카디널리티 정보는 로그·트레이스 또는 별도 저장소로 보냅니다.

쓰기 부하는 계속 높고, 읽기 부하는 대시보드와 경보 시점에 몰린다

시계열 데이터의 접근 패턴

Write path · 항상 높은 부하

모든 서버가 주기마다 샘플을 기록



최적화 방향 : 배치·압축·WAL·순차 쓰기·수평 샤딩

Read path · 짧고 큰 버스트

대시보드 새로고침·경보 평가가 동시에 발생



최적화 방향 : 최근 데이터 캐시·사전 집계·Query isolation

범용 데이터베이스보다 시계열 DB를 선택하는 것이 합리적이다

저장소 선택의 기준

비추천

관계형 DB

- 지속적인 고빈도 쓰기에 불리
- 레이블마다 별도 인덱스가 필요
- 이동 평균·시간 창 질의가 복잡
- 대규모 튜닝 비용이 높음

가능하지만 복잡

범용 NoSQL

- Cassandra·Bigtable로 구현 가능
- 파티션 키와 스키마 설계 난도가 높음
- 집계·보존 정책을 직접 구현
- 내부 구조에 대한 깊은 지식 필요

권장

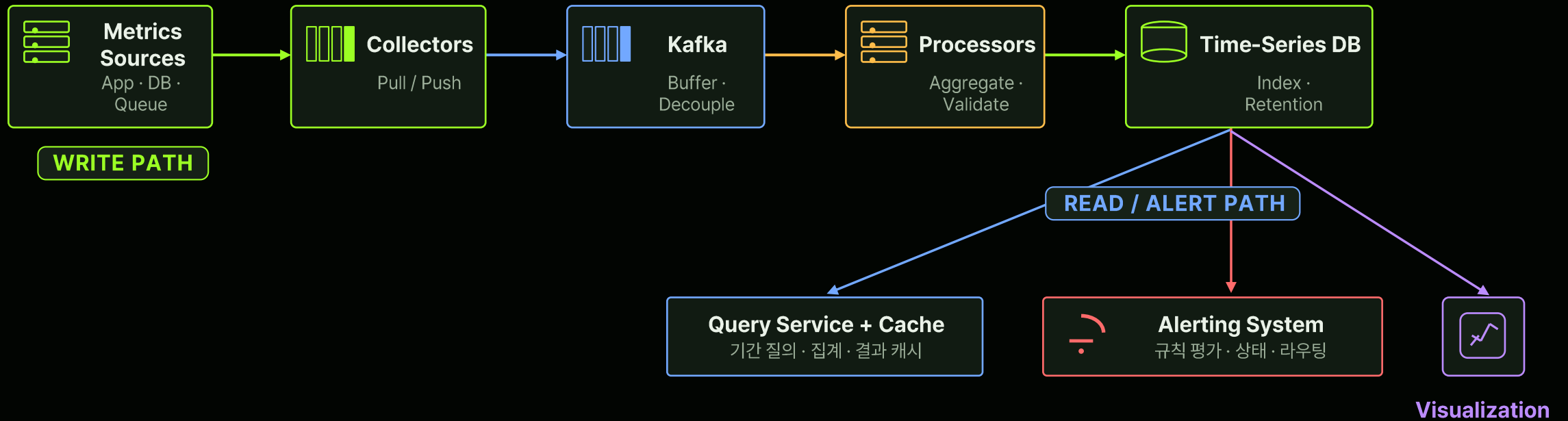
Time-Series DB

- 시계열 쓰기와 압축에 최적화
- 레이블 기반 빠른 탐색
- 전용 질의어와 시간 창 집계
- Retention·Downsampling 기능 제공

예: InfluxDB · Prometheus · OpenTSDB

수집 경로와 소비 경로를 분리한 파이프라인이 기본 구조다

개략적 아키텍처



지표 샘플과 경보 이벤트는 서로 다른 신뢰성 수준을 요구한다

Best-effort ingestion과 at-least-once alert delivery

Metric sample

CPU=71% @ 12:00:10

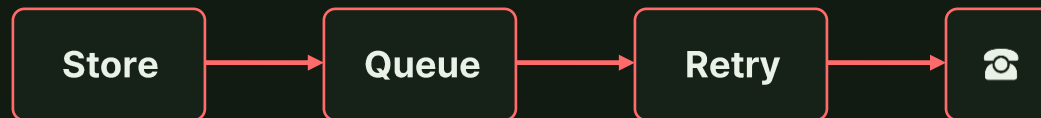


일부 샘플이 빠져도 전체 추세는 유지

- 카운터·CPU 사용률은 간헐적 누락을 허용할 수 있음
- 클라이언트는 fire-and-forget 방식으로 전송 가능
- 과도한 ACK·재전송은 수집 경로 비용을 높임

Alert event

Payment API error rate > 5%



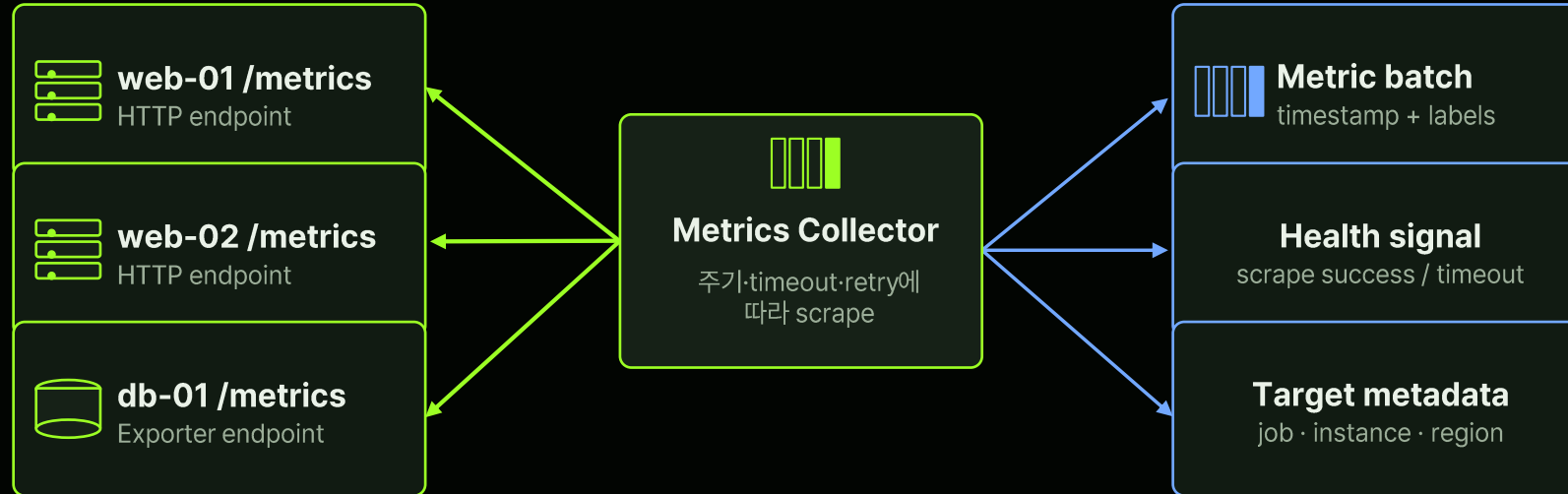
누락보다 중복이 낫다

- 상태 저장과 재시도로 최소 한 번 전달을 보장
- 중복 알림은 dedupe key로 억제
- 경보 채널 장애와 소비자 장애를 독립적으로 복구

Pull 모델에서는 Collector가 대상의 /metrics 엔드포인트를 주기적으로 읽는다

Collector 중심 수집 구조

PULL MODEL



장점

- 엔드포인트를 직접 열어 디버깅
- 응답 실패를 health check로 활용
- 수집 대상을 중앙 설정으로 통제

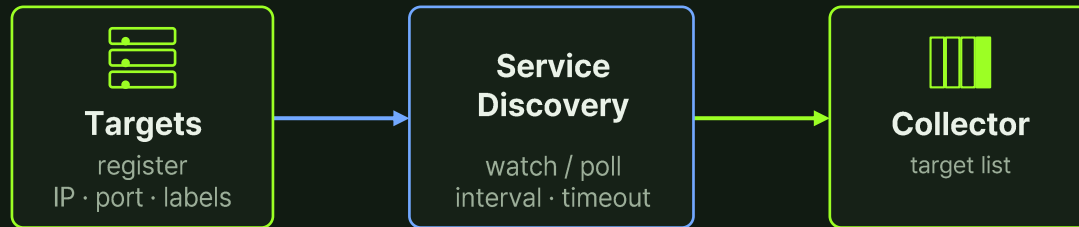
제약

- 대상 발견 필요
- 네트워크 접근성 필요

Service Discovery와 안정 해시가 Pull 수집기의 확장 문제를 해결한다

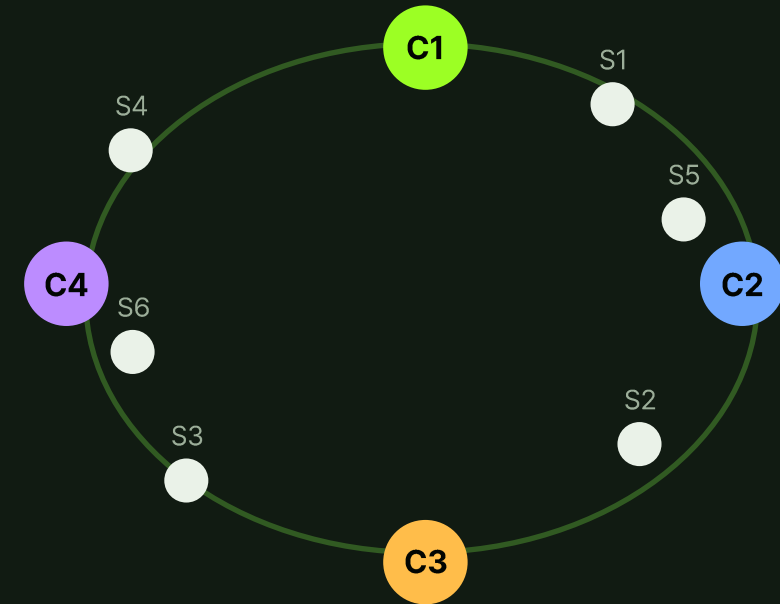
동적 대상 발견과 Collector 중복 방지

Service Discovery로 대상 목록 동기화



- 서버가 추가·삭제되면 대상 목록을 자동 갱신
- 수집 주기, timeout, retry 같은 메타데이터도 전달
- watch callback 또는 주기적 polling으로 변경 반영

안정 해시로 Collector 소유권 분배



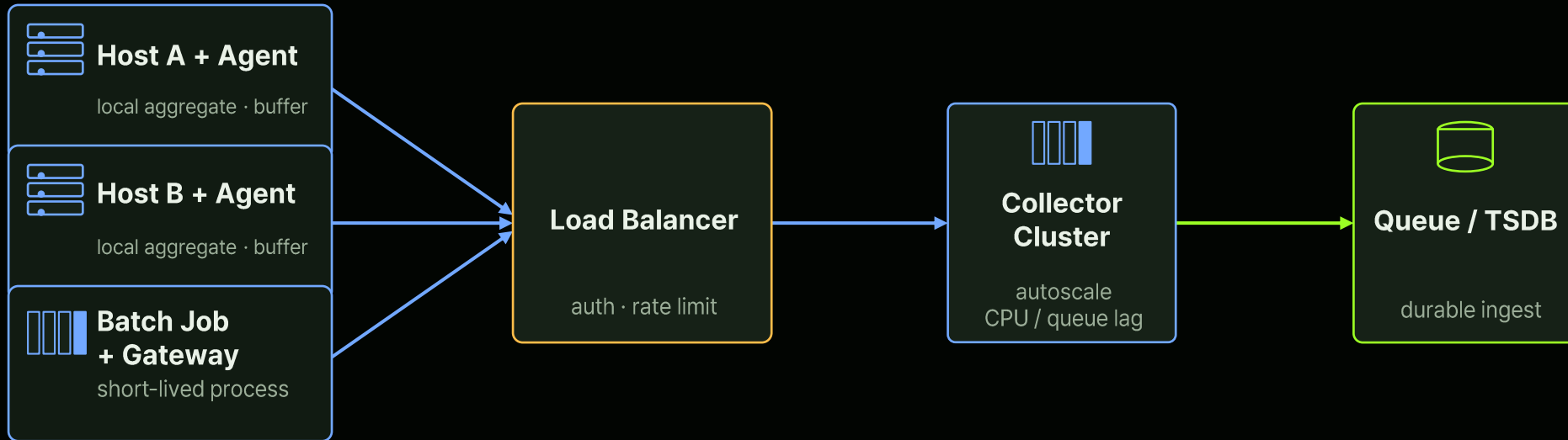
hash(target_id) → 다음 Collector

Collector 증감 시 일부 대상만 재배포

Push 모델에서는 각 서버의 Agent가 지표를 모아 Collector로 전송한다

Agent 중심 수집 구조

PUSH MODEL



주의점

Agent의 로컬 버퍼는
인스턴스가 종료되면
함께 사라질 수 있음

인증·허용 목록으로
가짜 지표 유입 방지

Agent에서 단순 카운터를 미리 집계하면 네트워크와 Collector 부하를 줄일 수 있다

Pull과 Push 중 절대적인 승자는 없고, 운영 환경에 따라 혼합한다

수집 모델 비교

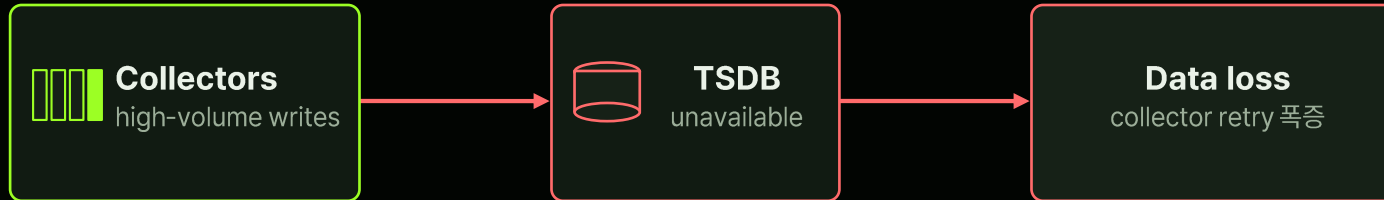
판단 기준	Pull	Push
디버깅	/metrics를 직접 확인	수신 실패 원인 분리가 어려움
Health check	Scrape 실패로 대상 장애 감지	네트워크와 대상 장애 구분 필요
Short-lived job	Push gateway가 필요	종료 전에 즉시 전송 가능
방화벽·멀티 DC	Collector가 모든 대상에 접근해야 함	중앙 endpoint로 보내기 쉬움
데이터 신뢰성	대상 목록이 통제되어 비교적 명확	인증·허용 목록이 필수

권장 결론 : 장기 실행 서비스는 Pull, short-lived·서버리스·복잡한 네트워크는 Push/Gateway를 지원하는 Hybrid

TSDB 앞의 Queue는 데이터 손실과 결합도를 줄이는 완충 계층이다

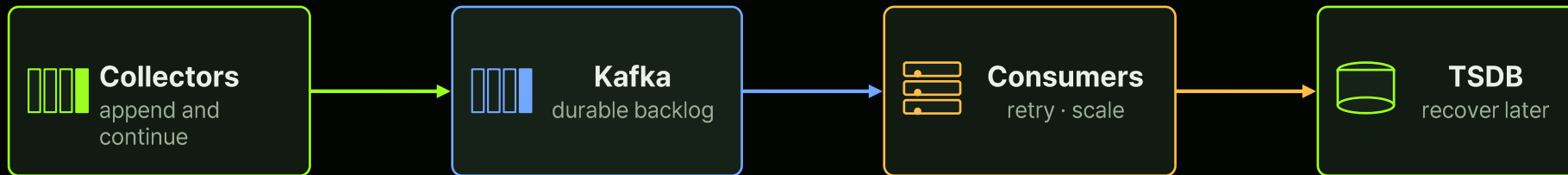
Kafka를 추가한 전송 파이프라인

Direct write



DB 장애가 수집 경로까지 전파

Queue-backed pipeline



효과

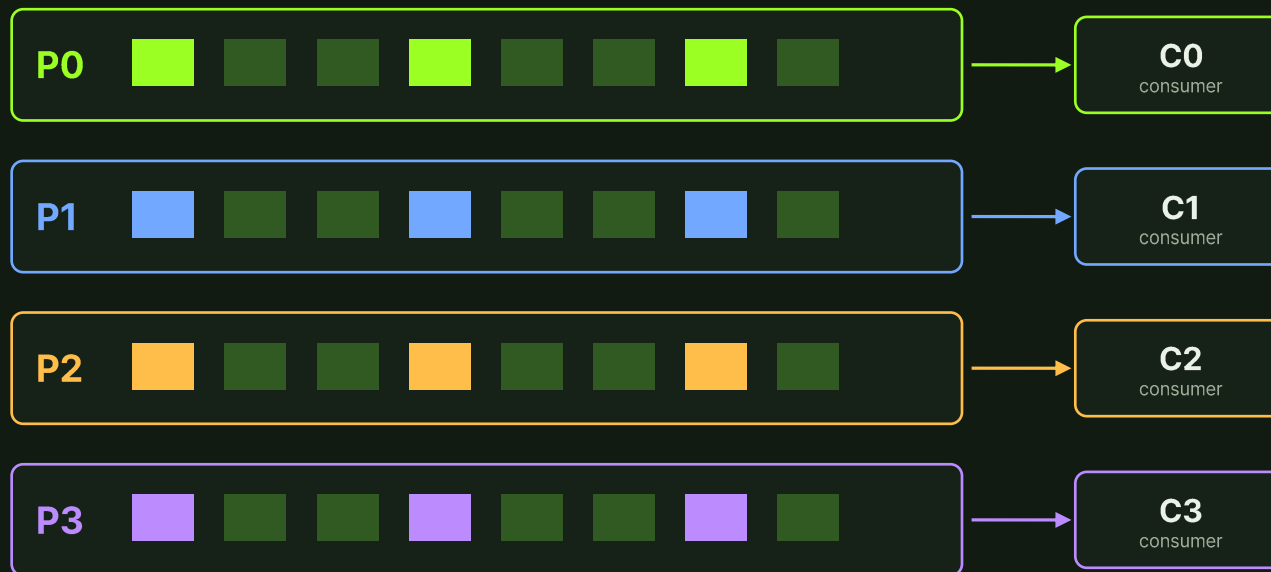
- 수집·처리 분리
- DB 장애 시 보존
- Consumer 독립 확장

Trade-off : Kafka 자체의 운영 복잡도와 추가 지연

파티션 키와 우선순위 정책이 처리 병렬성과 장애 영향 범위를 결정한다

Kafka를 통한 규모 확장

Partitioning by metric name + labels



같은 metric family를 같은 파티션에 두면 집계가 단순해짐

우선순위와 Hot Partition

P0 · paging-critical CPU down · error rate

P1 · dashboard latency · traffic

P2 · low priority debug metrics

- 고카디널리티 키는 특정 파티션을 과열시킬 수 있음
- 중요 지표를 별도 topic/consumer pool로 분리
- partition 수는 throughput과 consumer 병렬성에 맞춰 조정

집계를 앞당길수록 쓰기량은 줄지만 원본 정밀도와 재질의 유연성을 잃는다

집계가 일어날 수 있는 세 위치

1 Collection Agent 클라이언트 측

장점

- 전송량과 Collector 부하 감소
- 단순 counter·sum에 적합

비용

- 복잡한 집계는 어려움
- 인스턴스 종료 시 버퍼 손실

2 Ingestion Pipeline Flink · stream processor

장점

- DB 쓰기량 크게 감소
- 공통 roll-up을 중앙 관리

비용

- 늦은 이벤트 처리 필요
- 원본 정밀도와 재처리 유연성 감소

3 Query Side TSDB · Query service

장점

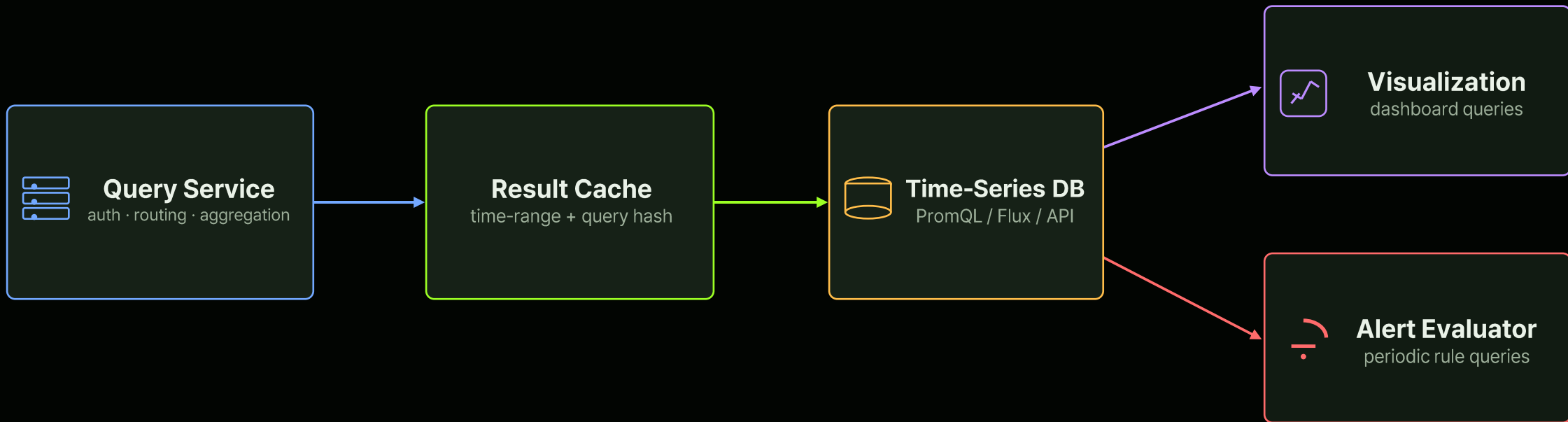
- 원본을 보존해 자유로운 분석
- 질의 시점에 다양한 집계 가능

비용

- 대규모 구간 질의가 느릴 수 있음
- 캐시·recording rule 필요

질의 계층은 소비자와 TSDB를 분리하지만, 불필요한 추상화가 될 수도 있다

Query server cluster와 결과 캐시



별도 Query Service가 필요한 경우

다중 TSDB 통합 · 공통 권한 · tenant 격리 · 쿼리 제한 · 결과 캐시

직접 연결이 나은 경우

Grafana·Alert 제품이 TSDB plugin과 cache를 이미 제공하고 구조가 단순할 때

시계열 전용 질의어는 "시간 창 + 레이블 집계"를 직접 표현한다

지난 10분간 us-west 웹 서버 CPU 평균 질의

질문

지난 10분간 us-west 지역
모든 웹 서버의 CPU 평균은?

선택 조건

- metric = cpu_load
- region = us-west
- range = 10m
- aggregate = avg by region

범용 SQL 관점

```
SELECT AVG(value)
FROM metrics
WHERE name = 'cpu_load'
      AND region = 'us-west'
      AND ts > NOW() - 10m
GROUP BY time_bucket, host;
```

스키마·윈도 함수·버킷 처리를 직접 구성

PromQL 예시

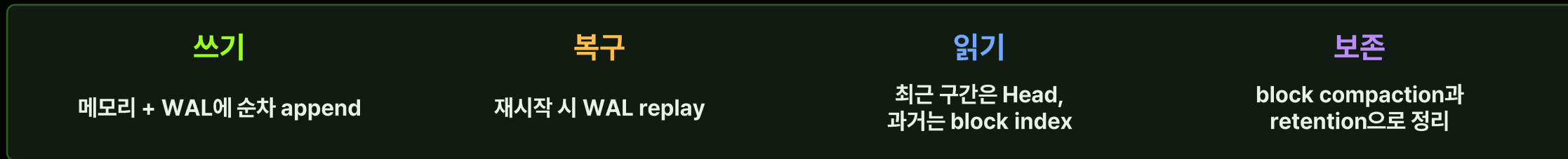
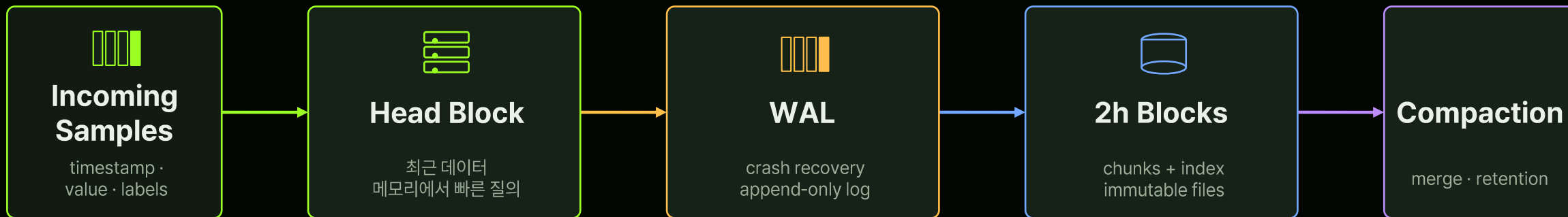
```
avg by (region) (
  avg_over_time(
    cpu_load{region="us-west"}[10m]
  )
)
```

레이블 필터 + range vector
+ 집계를 직접 표현

대시보드와 경고 규칙에서 재사용

최근 샘플은 메모리에서 받고, WAL과 불변 블록으로 내구성과 읽기 성능을 확보한다

Prometheus 계열 저장 경로의 개념적 구조



절대값 대신 변화량을 저장하면 타임스탬프와 값의 압축률이 크게 올라간다

데이터 인코딩과 압축

Timestamp delta encoding



반복되는 32/64비트 절대 타임스탬프 대신 작은 변화량을 저장

TSDB는 값·레이블·chunk에도 전용 압축을 적용

압축이 잘 되는 이유

규칙적 시간 간격 10s · 30s · 60s처럼 반복

작은 값 변화 CPU·latency가 인접 샘플에서 유사

반복 레이블 metric·host·region 문자열을 사전화

불변 블록 block 단위로 압축·compaction 가능

좋은 TSDB는 이 최적화를 저장 엔진에 기본 제공한다

오래된 데이터의 해상도를 낮추면 1년 보존 비용을 수십 배 줄일 수 있다

Downsampling · Roll-up · Cold storage

Retention pyramid

0-7일 · 원본 10초 해상도

60,480 samples / series

8-30일 · 1분 해상도

33,120 samples / series

31일-1년 · 1시간 해상도

8,040 samples / series

Cold storage / archive

rarely queried

샘플 수 비교

3.15M

10초 해상도로 1년 유지

samples / series

101.6K

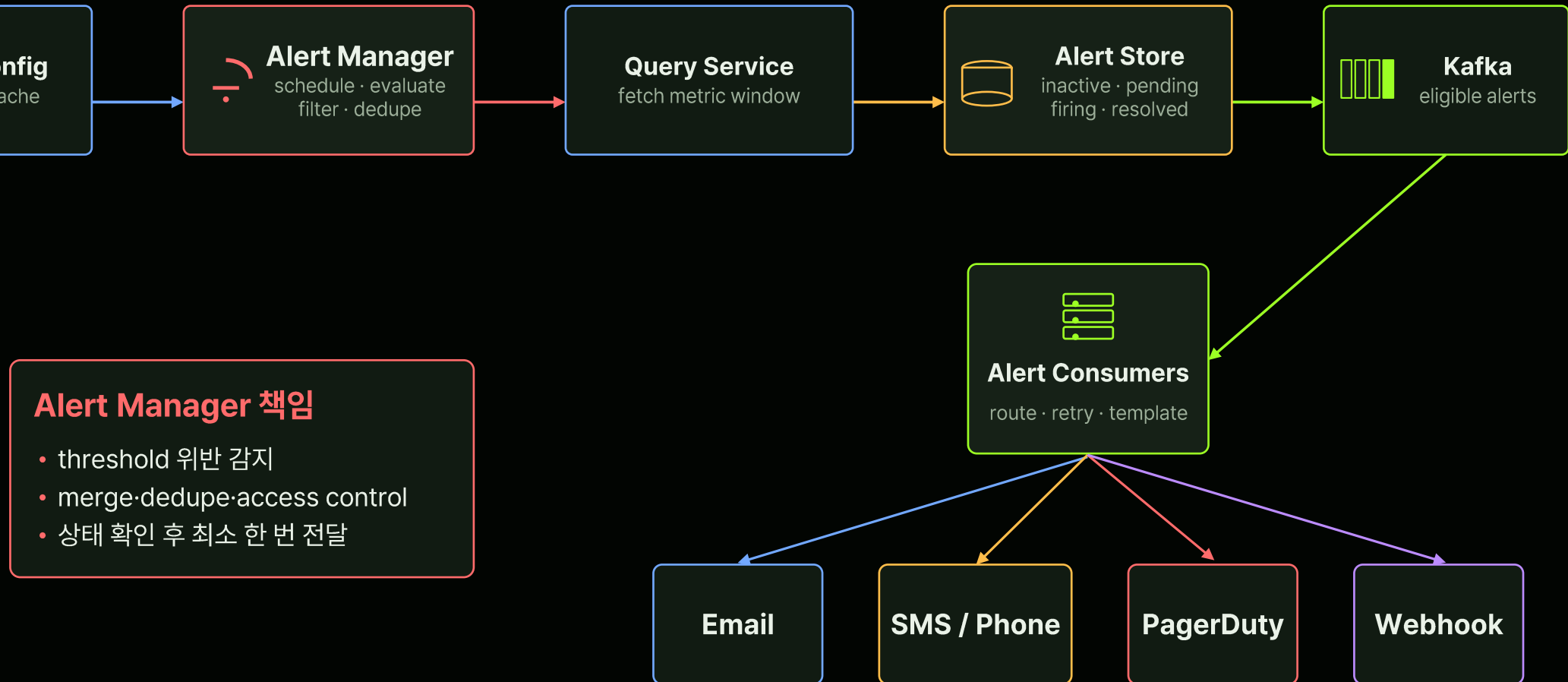
계층형 보존 정책

samples / series

약 31× fewer samples

경보 시스템은 규칙 평가와 알림 전송을 분리하고 상태를 영속화한다

Rule evaluation → alert event → notification



좋은 경보는 빨리 울리는 것보다 “중복 없이 행동 가능하게” 울리는 것이 중요하다

상태 전이와 경보 노이즈 제어

Alert lifecycle



순간 스파이크는 Pending에서 사라지고, 조건이 지속될 때만 Firing

상태를 영속화하면 평가기 재시작 후에도
for duration과 재시도를 복구할 수 있다

Noise control

Grouping 같은 service-cluster의 알림을 한 건으로 묶음

Deduplication 동일 fingerprint의 반복 이벤트를 억제

Silence 점검 시간 동안 조건에 맞는 알림을 일시 중단

Inhibition cluster-down이 올리면 하위 instance 알림을 억제

목표: “누가 무엇을 해야 하는지”가 명확한 경보만 사람에게 전달

경보와 시각화는 직접 구축보다 검증된 제품을 통합하는 편이 대개 낫다

Build vs Buy

Dashboard mock

Production Overview

LAST 1H

RPS

82.4K

P99

241ms

ERROR

1.7%

Request rate by region

의사결정 기준

직접 구축 제품 차별화·특수 UX·엄격한 tenant 정책

제품 활용 표준 대시보드·채널 통합·플러그인 생태계

운영 비용 권한·템플릿·알림 라우팅·업그레이드까지 포함

권장 수집·저장에 집중하고 시각화/경보는 구매·통합

예: Grafana + Alertmanager / PagerDuty

모니터링 시스템도 장애가 나므로 각 단계의 실패 모드와 복구 경로를 설계해야 한다

주요 장애 시나리오와 대응

Collection 장애

Collector down hash ring 재할당·수평 확장

Target churn Service Discovery watch

Push overload Agent buffer·LB·rate limit

Storage / Query 장애

TSDB unavailable Kafka backlog로 흡수

Query spike cache·limit·isolation

Old data loss replica·cold archive

Alerting 장애

Evaluator restart state 복구·rule HA

Channel failure retry·fallback route

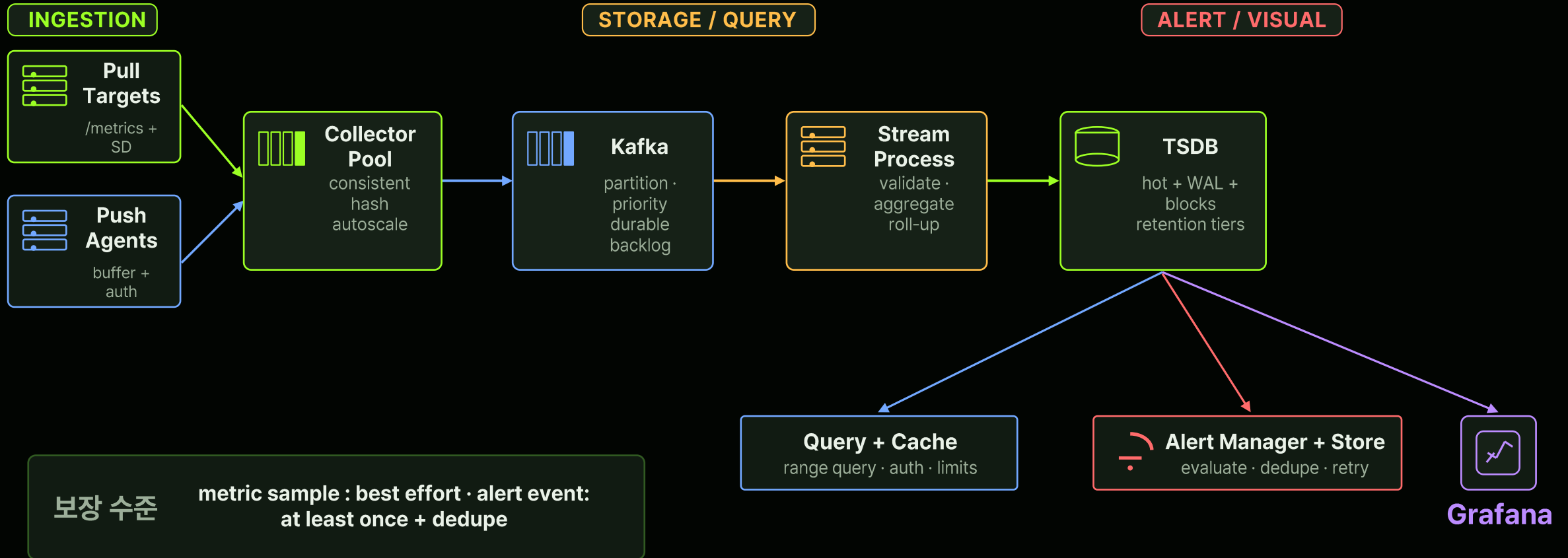
Alert storm grouping·inhibition

Self-monitoring + external heartbeat

Collector lag · Kafka lag · rule evaluation duration · notification failure를 별도 시스템 또는 외부 heartbeat에서 감시

최종 설계는 "혼합 수집 + Queue 완충 + 계층형 TSDB + 상태 기반 경보"로 정리된다

상세 설계를 반영한 최종 아키텍처



이 설계의 핵심은 기술 이름보다 데이터 특성에 맞는 경계를 선택하는 것이다

핵심 요약

1

Scope

운영 지표만 포함하고 로그·트레이싱은 제외

2

Data model

metric name + labels + timestamp/value

3

Collection

Pull/Push의 장단점을 이해하고 Hybrid 지원

4

Scale

Collector pool + Kafka + partitioning

5

Storage

TSDB + 압축 + downsampling + cold tier

6

Alerting

상태 저장 + dedupe + retry + 채널 분리

한 문장 요약 : “대량 시계열을 저비용으로 보존하고, 중요한 이상만 신뢰성 있게 사람에게 전달하는 파이프라인”

스터디에서는 "정답"보다 선택 조건과 실패 시나리오를 토론해 보자

설계 토론 질문 3개

Q1

우리 조직이 수집 모델을 하나만 선택해야 한다면 Pull과 Push 중 무엇을 고를 것인가?

Q2

TSDB가 30분간 중단될 수 있다면 Kafka를 반드시 넣어야 할까, 아니면 TSDB의 HA와 local buffer로 충분할까?

Q3

경보 전달은 at-least-once + dedupe로 충분한가? 중복 알림과 누락 중 어느 쪽을 더 허용할 수 있는가?